

PROJECT

3 - TIER ARCHITECTURE With Public and Private Machines

Name: Krishna Teja Meduri

INTRODUCTION:

Cloud computing has become the foundation of modern application deployment due to its scalability, security, and cost-effectiveness. A 3-tier architecture separates an application into three independent layers: the presentation layer, application layer, and database layer. This separation improves performance, security, and maintainability.

In this project, an AWS-based 3-tier architecture was implemented using a Virtual Private Cloud (VPC). The web server hosted Nginx as a reverse proxy in a public subnet, while the application server running Apache Tomcat and the MySQL database server were deployed in private subnets. Internet access for the private servers was provided securely through a NAT Gateway.

PROBLEM STATEMENT:

Traditional application deployment on a single server creates several challenges, such as limited scalability, security vulnerabilities, and difficulty in maintenance. Direct exposure of application and database servers to the internet increases the risk of cyberattacks and unauthorized access. The challenge is to design a secure and scalable cloud architecture where only the web server is publicly accessible, while the application and database servers remain isolated in private subnets without compromising connectivity and functionality.

OBJECTIVES:

The main objectives of this project are:

- Design and implement a secure 3-tier architecture using AWS.
- Create a Virtual Private Cloud (VPC) with public and private subnets.
- Deploy an Nginx Web Server in the public subnet.
- Deploy an Apache Tomcat Application Server in a private subnet.
- Deploy a MySQL Database Server in another private subnet.
- Configure Nginx as a reverse proxy to forward requests to the Tomcat application.
- Enable internet access for private instances using a NAT Gateway.
- Secure communication using AWS Security Groups.
- Verify connectivity between the Application Server and Database Server using Telnet.

PROPOSED SOLUTION:

The proposed solution implements a secure AWS infrastructure using a 3-tier architecture.

- The Web Server is deployed in the public subnet with a public IP address.
- The Application Server and Database Server are deployed in private subnets without public IP addresses.
- An Internet Gateway allows internet access only to the public subnet.
- A NAT Gateway enables private instances to download software updates without exposing them to the internet.
- Nginx acts as a reverse proxy, forwarding incoming HTTP requests to the Tomcat Application Server.
- MySQL stores application data and is accessible only from the Application Server through Security Group rules.

This architecture improves security, scalability, and availability while following AWS best practices.

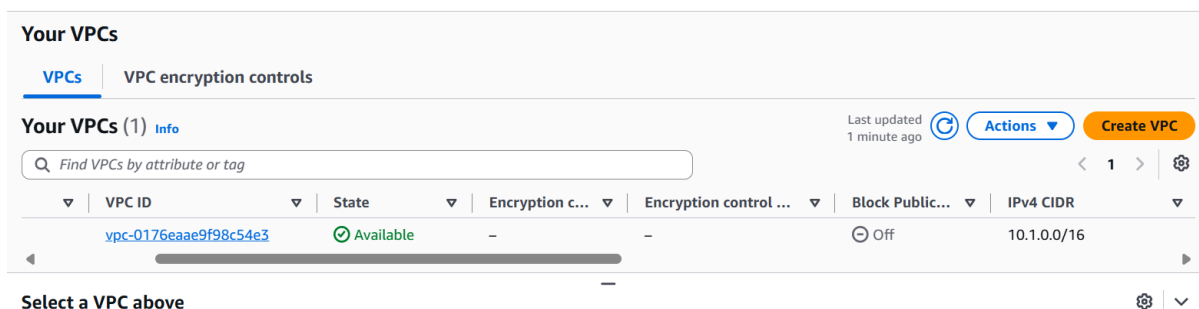
TECHNOLOGIES USED:

- Amazon Web Services (AWS) is a cloud platform used to deploy and manage the complete infrastructure.
- Amazon EC2 (Elastic Compute Cloud) – Virtual machines used for the Web Server, Application Server, and Database Server.
- Amazon VPC (Virtual Private Cloud) – Created an isolated network environment for secure communication.
- Public Subnet – Hosted the Nginx Web Server with a public IP address.
- Private Subnets – Hosted the Apache Tomcat Application Server and MySQL Database Server without public IPs.
- Internet Gateway (IGW) – Enabled internet access for resources in the public subnet.
- NAT Gateway – Provided secure internet access to instances in private subnets for software installation and updates.
- Elastic IP (EIP) – Assigned to the NAT Gateway for internet connectivity.
- Route Tables – Managed network traffic between subnets, Internet Gateway, and NAT Gateway.
- Security Groups – Configured firewall rules to control inbound and outbound traffic.
- Nginx is configured as a reverse proxy web server to forward client requests to the application server.
- Apache Tomcat 11 – Java application server used to host the web application.
- OpenJDK 17 – the Java Development Kit required to run Apache Tomcat.
- MySQL Server – a relational database management system used for backend data storage.
- MobaXterm – an SSH client used to connect to and manage EC2 instances remotely.
- SSH (Secure Shell) is used for secure remote access to Linux instances.
- Telnet is used to verify network connectivity between the Application Server and Database Server on port 3306.
- Linux Command Line – Used for server configuration, package installation, and service management.

IMPLEMENTATION STEPS:

Step 1: Create an AWS VPC

- Created a VPC with CIDR block 10.1.0.0/16.



Step 2: Create Subnets

Created three Subnets

- Web Subnet (Nginx) - Public-1
- App Subnet (Tomcat) - Private-1
- Database Subnet (MySQL) - Private-2

Subnets (3) [Info](#) Last updated 4 minutes ago [Actions](#) [Create subnet](#)

Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC	Block Public...
☐	private-1	subnet-016546223c331bd3	Available	vpc-0176eaae9f98c54e3 vpc	Off
☐	public-1	subnet-04b5c16a5e6372802	Available	vpc-0176eaae9f98c54e3 vpc	Off
☐	private-2	subnet-08a0bba4eca444bd7	Available	vpc-0176eaae9f98c54e3 vpc	Off

Select a subnet

Step 3: Create Internet Gateway

- Created an Internet Gateway.
- Attached it to the VPC.

Internet gateways (1) [Info](#) Last updated 4 minutes ago [Actions](#) [Create internet gateway](#)

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State	VPC ID	Owner
☐	ig	igw-0ac091f93f418ef14	Attached	vpc-0176eaae9f98c54e3 vpc	73539

Select an internet gateway above

Step 4: Configure Route Tables

The default Route Table is updated with a route (0.0.0.0/0) pointing to the Internet Gateway, allowing outbound internet connectivity.

Route tables (1) [Info](#) Last updated 10 minutes ago [Actions](#) [Create route table](#)

Find route tables by attribute or tag

Name = [drt](#) [Clear filters](#)

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	drt	rtb-0fee70cd572b67f96	-	-	Yes	vpc-0176eaae9f98c54e3 vpc

Select a route table

Create a Private route table and associate the two private subnets with the private route table.

Route tables (2) [Info](#) Last updated 11 minutes ago [Actions](#) [Create route table](#)

Find route tables by attribute or tag

Name = [drt](#) [Name = private-rt](#) [Clear filters](#)

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	drt	rtb-0fee70cd572b67f96	-	-	Yes	vpc-0176eaae9f98c54e3 vpc
☐	private-rt	rtb-0b5b3db67a52880d1	2 subnets	-	No	vpc-0176eaae9f98c54e3 vpc

Select a route table

Step 5: Create a NAT gateway and attach it to the private route table.

NAT gateways (1) [Info](#)

Find NAT gateways by attribute or tag

Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route
natgateway	nat-1574f61e548ee9159	Public	Available	-	Regional	rtb-0a1

Select a NAT gateway

Add a default route (0.0.0.0/0) that points to the NAT Gateway.

rtb-0b5b3db67a52880d1 / private-rt

Details [Info](#)

Route table ID rtb-0b5b3db67a52880d1	Main No	Explicit subnet associations 2 subnets	Edge associations -
VPC vpc-0176eaae9f98c54e3 vpc	Owner ID 735391218186		

Routes (2) [Both](#) [Edit routes](#)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-1574f61e548ee9159	Active	No	Create Route
10.1.0.0/16	local	Active	No	Create Route Table

Step 6:

Create a Web-Server EC2 instance and attach the public subnet.

Create an app-Server EC2 instance and attach a private subnet.

Create a Database-Server EC2 instance and attach a private subnet.

Instances (3) [Info](#)

Last updated about 2 hours ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets [Choose filter set](#) Find Instance by attribute or tag (case-sensitive)

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DI
☐	app	i-0fb04252ff4c3c34	Running	t3.micro	Initializing	us-east-2b	-
☐	db	i-0936a278b75369d17	Running	t3.micro	Initializing	us-east-2b	-
☐	web	i-0676adaead1391867	Running	t3.micro	3/3 checks passed	us-east-2b	-

Select an instance

Step 7:

In security groups inbound rules: add SSH, HTTP, MySQL

VPC > Security Groups [sg-05d57abf202a5fb86](#)

AWS Global View

Virtual private cloud

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers
- Security**
 - Network ACLs
 - [Security groups](#)

Security group details:

- Security group name:** default
- Security group ID:** [sg-05d57abf202a5fb86](#)
- Owner:** [735391218186](#)
- Inbound rules count:** 5 Permission entries
- Outbound rules count:** 1 Permission entry
- Description:** default VPC security group
- VPC ID:** [vpc-0176eaae9f98c54e3](#)

Inbound rules (5) [Manage tags](#) [Edit inbound rules](#)

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0d3c2082b6b65afdc	IPv4	MYSQL/Aurora	TCP	3306
☐	-	sgr-0553aefe797ce7ee8	IPv4	SSH	TCP	22
☐	-	sgr-0afc6e5ccb63200d3	IPv4	Custom TCP	TCP	8080
☐	-	sgr-09aa1fdfeadea376f9	IPv4	HTTP	TCP	80
☐	-	sgr-00f95b89ac45f9895	IPv4	All traffic	All	All

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Step 8:

Connect to the web server

root@ip-10.1.1.187:/home/ubuntu

cmd: apt update

cmd: apt install nginx

cmd: apt install nginx -y

Check whether Nginx is installed on our web server.



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Nginx is installed on our web server.

Step 9:

Connect to app-ser from web-ser

Cmd: chmod 400 key-name.pem

Cmd: ssh -i keyname.pem ubuntu@<app-server-private-ip>

root@ip-10.1.2.14:/home/ubuntu#

cmd: sudo apt update

cmd: apt install default-jdk -y

cmd: wget https://dlcdn.apache.org/tomcat/tomcat-11/v11.0.23/bin/apache-tomcat-11.0.23.tar.gz

cmd: ls -lt

cmd: tar -xvzf apache-tomcat-11.0.23.tar.gz

cmd: ls -lt

cmd: mv apache-tomcat-11.0.23 tomcat

cmd: ls

cmd: cd tomcat/bin/

cmd: ls

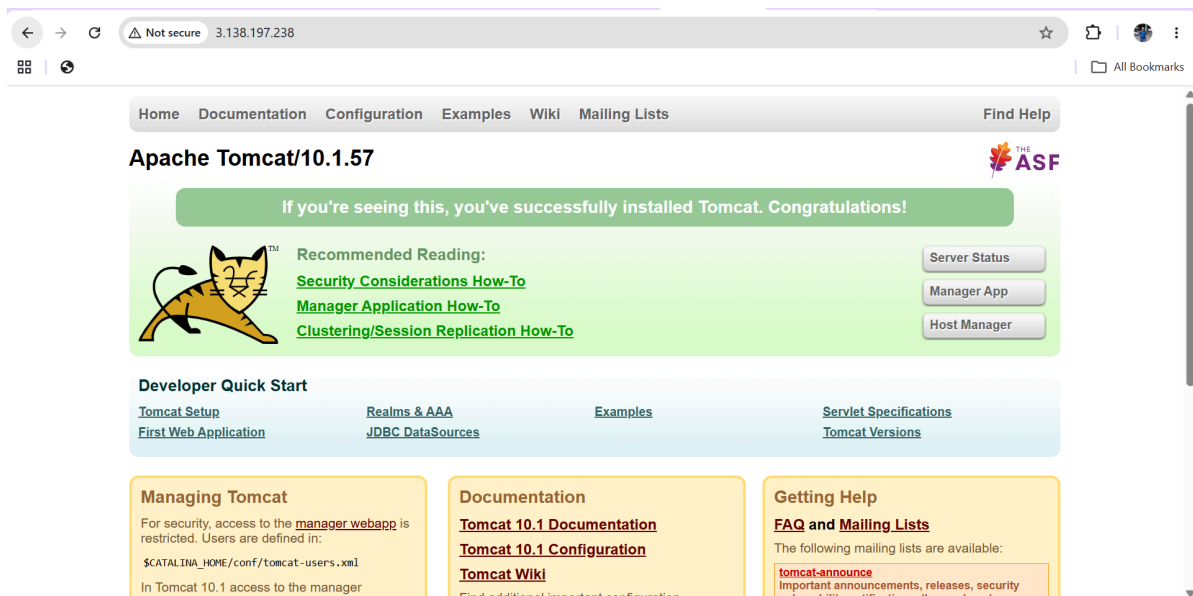
cmd: ./startup.sh

To Check Whether Tomcat is installed or not in our app server.

In web-server: sudo nano /etc/nginx/sites-available/default

Go to location/ and replace it with:

```
listen 80; location /
{
    proxy_pass http://10.1.2.14:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```



Tomcat is installed in our app server.

Step 10:

Connect to db-ser from web-ser

Cmd: `chmod 400 key-name.pem`

Cmd: `ssh -i keyname.pem ubuntu@<app-server-private-ip>`

`root@ip- 10.1.3.198:/home/ubuntu#`

Cmd: `apt update`

Cmd: `apt install mysql-server -y`

Cmd: `mysql_secure_installation`

➤ Edit the MySQL configuration file using a text editor:

`# vi /etc/mysql/mysql.conf.d/mysqld.cnf`

➤ Locate the following line: `bind-address = 127.0.0.1` • `bind-address = 0.0.0.0`

• Save and exit (CTRL + X, then Y, then Enter).

Cmd: `systemctl restart mysql`

Cmd: `systemctl status mysql`

Check whether MYSQL is installed or not on our database server.

```
Reloading the privilege tables will ensure that all changes
made so far will take effect immediately.

Reload privilege tables now? (Press y|Y for Yes, any other key for No) : yea
Success.

All done!
root@ip-10-1-3-198:/home/ubuntu# vi /etc/mysql.conf.d/mysqld.cnf
root@ip-10-1-3-198:/home/ubuntu# vi /etc/mysql.conf.d/mysqld.cnf
root@ip-10-1-3-198:/home/ubuntu# systemctl restart mysql
Command 'system' not found, did you mean:
  command 'system3' from deb simh (3.8.1-6.3)
Try: apt install <deb name>
root@ip-10-1-3-198:/home/ubuntu# systemctl restart mysql
root@ip-10-1-3-198:/home/ubuntu# systemctl status mysql
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-07-13 11:02:46 UTC; 24s ago
     Invocation: 525b115532094b67a2da410fc2c51e08
   Process: 2216 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
    Main PID: 2226 (mysqld)
      Status: "Server is operational"
        Tasks: 35 (limit: 627)
       Memory: 500.7M (peak: 500.8M)
          CPU: 942ms
       CGroup: /system.slice/mysql.service
               └─2226 /usr/sbin/mysqld

Jul 13 11:02:44 ip-10-1-3-198 systemd[1]: Starting mysql.service - MySQL Community Server...
Jul 13 11:02:46 ip-10-1-3-198 systemd[1]: Started mysql.service - MySQL Community Server.
root@ip-10-1-3-198:/home/ubuntu#
```

MYSQL is installed on our database server

Step 11:

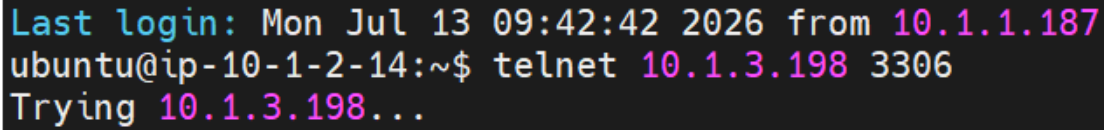
Now connect the web server to the app server.

Open web server

Cmd: `chmod 400 key-name.pem`

Cmd: `ssh -i keyname.pem ubuntu@<app-server-private-ip>`

Cmd: `telnet 10.1.3.198 3306`



```
Last login: Mon Jul 13 09:42:42 2026 from 10.1.1.187
ubuntu@ip-10-1-2-14:~$ telnet 10.1.3.198 3306
Trying 10.1.3.198...
```

The Database server is connected to the App server

Conclusion:

The AWS 3-Tier Architecture project successfully demonstrates how to build a secure, scalable, and reliable infrastructure for hosting a web application in the cloud. By separating the application into the Web Tier, Application Tier, and Database Tier, the architecture improves security, performance, and maintainability while following AWS best practices. The Web Server is deployed in a public subnet to handle incoming user requests, while the Application Server and Database Server are deployed in private subnets to prevent direct internet access. An Internet Gateway enables communication with the public subnet, and a NAT Gateway provides secure outbound internet access for private instances. Route tables and security groups ensure that communication between the three tiers is controlled and secure. This architecture provides several advantages, including enhanced security through network isolation, improved scalability by allowing each tier to scale independently, higher availability, easier maintenance, and better overall performance. It also serves as a strong foundation for deploying enterprise applications on AWS.